

K-디지털 트레이닝

MyBatis 연동 프로그래밍

[KB] IT's Your Life

✓ 프로젝트 생성

- 프로젝트명: mybatis-ex
- Gradle/Groovy
- 그룹 ID: org.scoula
- 아티팩트 ID: mybatis-ex

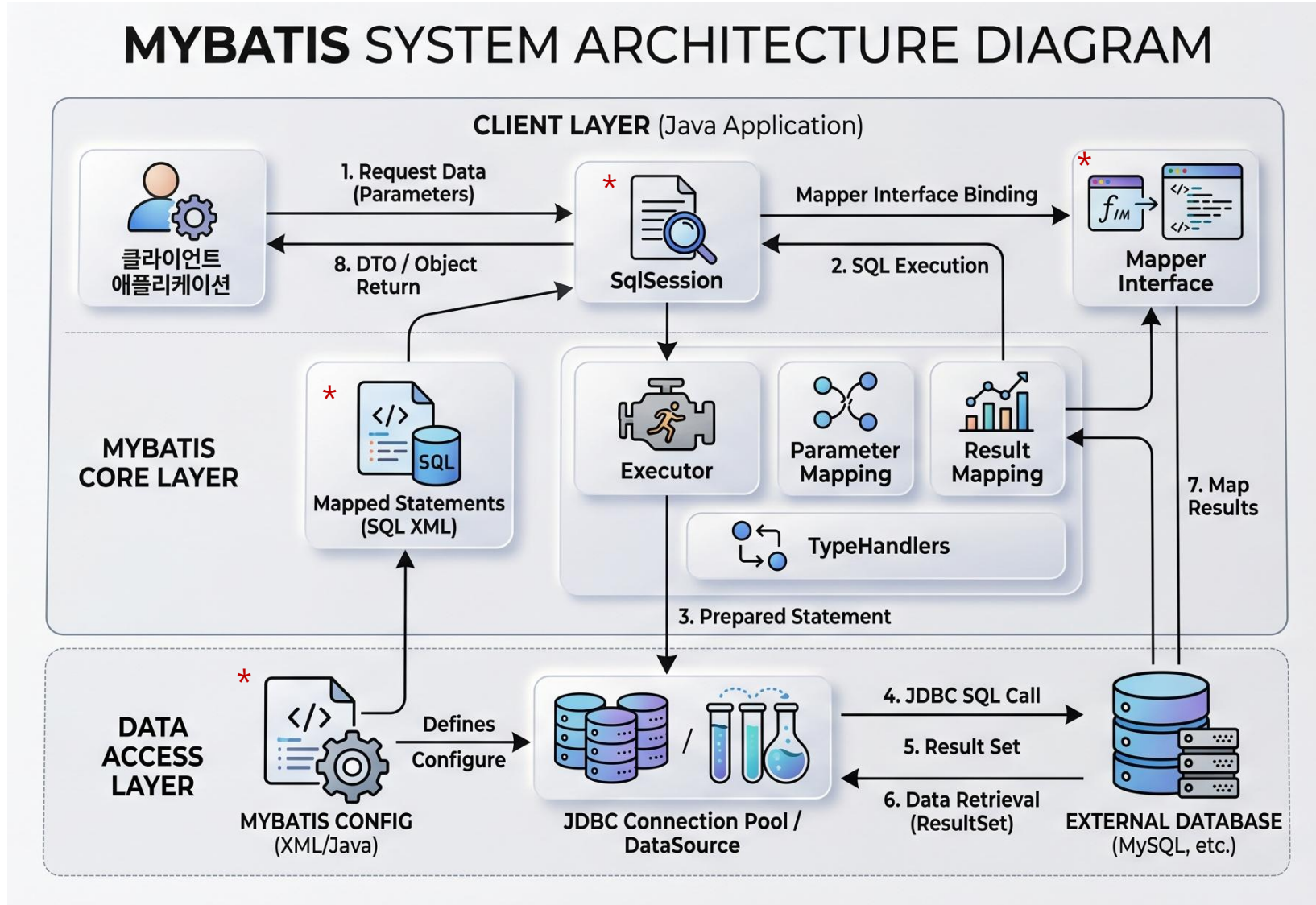
build.gradle

```
dependencies {  
    // MySQL jdbc 드라이버  
    implementation 'com.mysql:mysql-connector-j:8.3.0'  
  
    // MyBatis 라이브러리  
    implementation 'org.mybatis:mybatis:3.5.13'  
  
    compileOnly 'org.projectlombok:lombok:1.18.30'  
    annotationProcessor 'org.projectlombok:lombok:1.18.30'  
  
    testCompileOnly 'org.projectlombok:lombok:1.18.30'  
    testAnnotationProcessor 'org.projectlombok:lombok:1.18.30'  
  
    testImplementation platform('org.junit:junit-bom:5.10.0')  
    testImplementation 'org.junit.jupiter:junit-jupiter'  
}
```

✓ MyBatis

- 자바 언어를 위한 오픈 소스 퍼시스턴스 프레임워크
- 객체 지향 프로그래밍 언어와 관계형 데이터베이스 간의 불일치를 해결하는 역할
- 주요 특징
 - SQL 매핑
 - XML 파일이나 애너테이션을 사용하여 자바 메서드와 SQL 문을 매핑
 - 자바 코드에서 직접 SQL 문을 작성하는 번거로움을 줄이고, SQL 문을 더욱 편리하게 관리
 - 동적 SQL 지원
 - 동적 SQL을 지원하여, 런타임 시 조건에 따라 SQL 문을 변경
 - 더욱 유연하고 강력한 데이터 액세스 로직을 구현
 - 캐싱 지원
 - 캐싱을 지원하여, 빈번하게 사용되는 데이터를 메모리에 저장하여 데이터베이스 액세스 속도를 향상
 - 트랜잭션 관리
 - 트랜잭션 관리를 지원하여, 여러 데이터베이스 작업을 하나의 트랜잭션으로 묶어 데이터 일관성을 유지

MyBatis 아키텍처



resources/application.properties

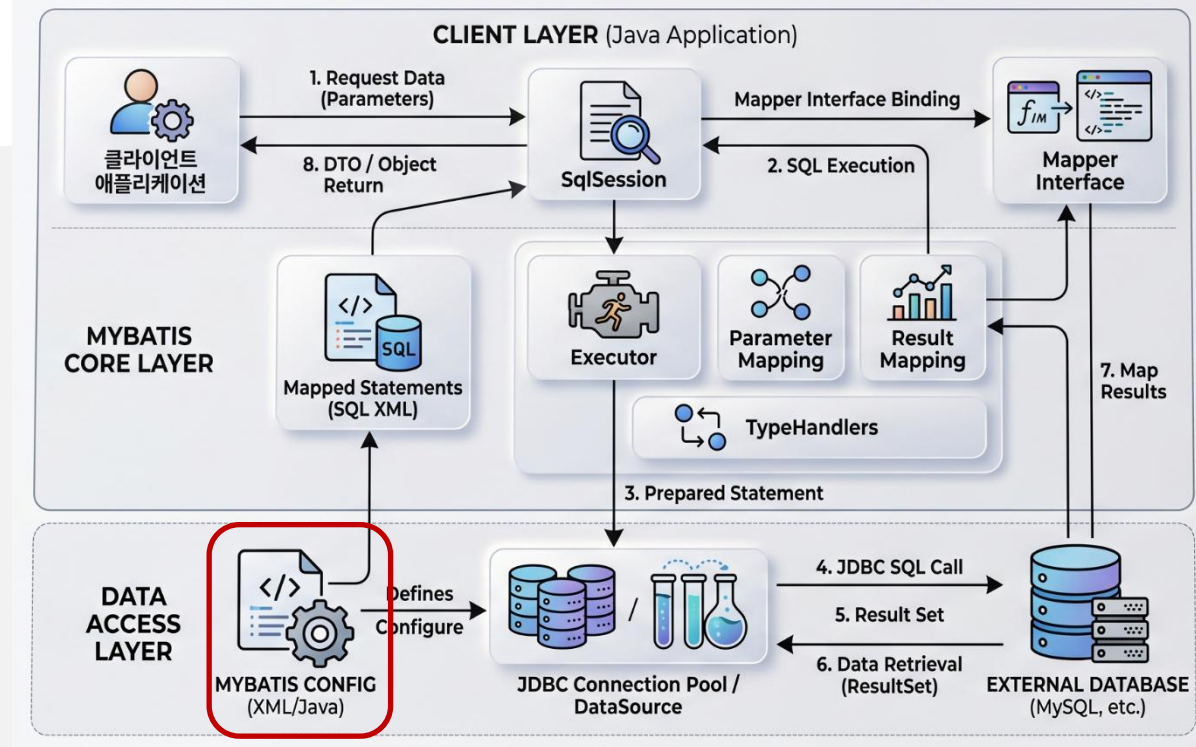
```
driver=com.mysql.cj.jdbc.Driver  
url=jdbc:mysql://127.0.0.1:3306/jdbc_ex  
username=scoula  
password=1234
```

resources/mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
  PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
  <properties resource="application.properties" />

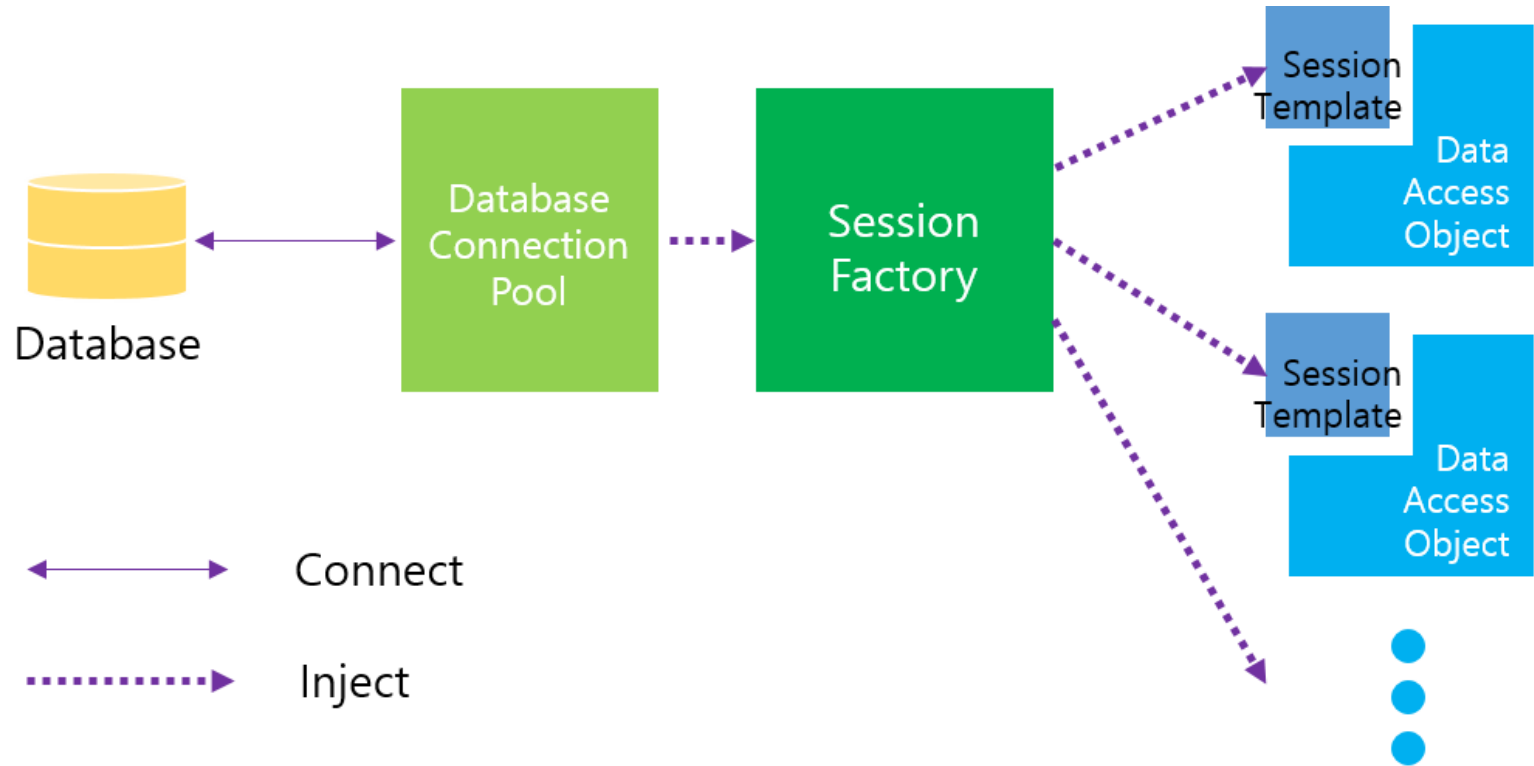
  <settings>
    <setting name="cacheEnabled" value="false" />
    <setting name="mapUnderscoreToCamelCase" value="true" />
    <setting name="useGeneratedKeys" value="false" />
  </settings>

  <environments default="development">
    <environment id="development">
      <transactionManager type="JDBC" />
      <dataSource type="POOLED">
        <property name="driver" value="${driver}" />
        <property name="url" value="${url}" />
        <property name="username" value="${username}" />
        <property name="password" value="${password}" />
      </dataSource>
    </environment>
  </environments>
</configuration>
```



✓ MyBatis 주요 객체

- **SqlSessionFactoryBuilder**
 - MyBatis 설정을 읽어 SqlSessionFactory 객체 생성
- **SqlSessionFactory**
 - SqlSession 객체 생성 및 관리
- **SqlSession**
 - 데이터베이스와 연결된 작업공간 관리 객체
 - SQL문 실행 및 트랜잭션 관리



org.scoula.database.MyBatisConfig.java

```
package org.scoula.database;
```

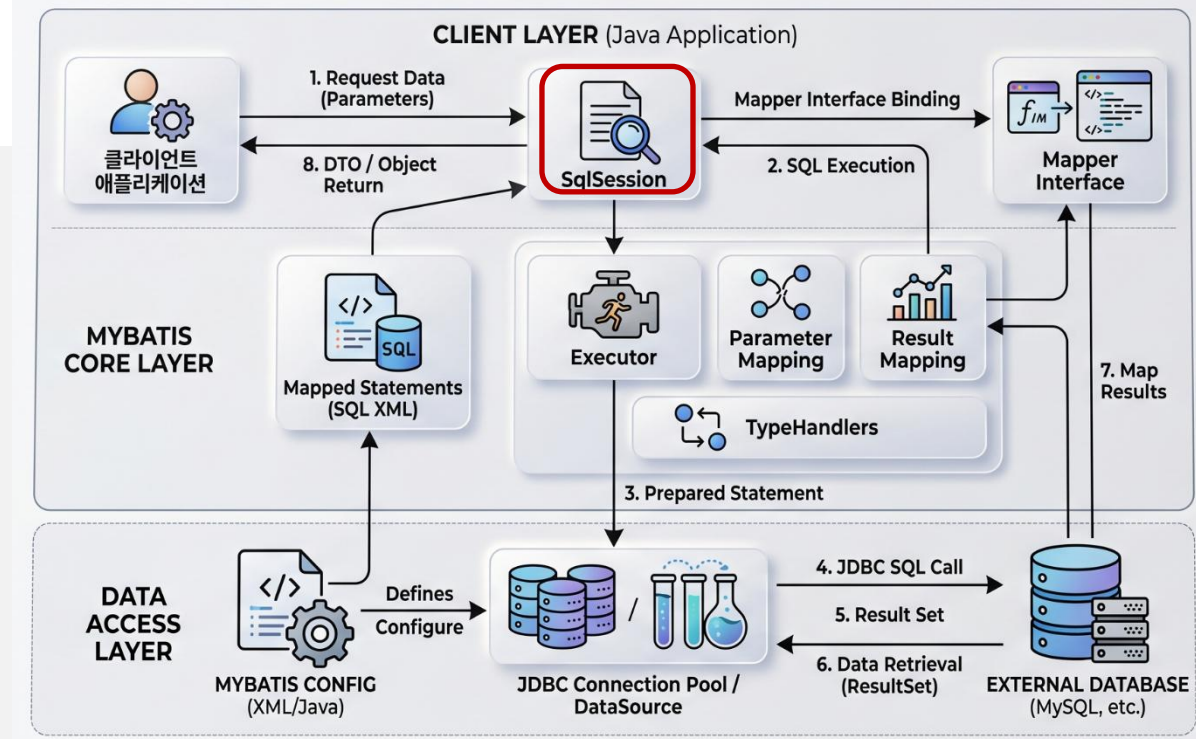
```
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
```

```
import java.io.Reader;
```

```
public class MyBatisConfig {
    private static SqlSessionFactory sqlSessionFactory;

    static {
        try {
            Reader reader = Resources.getResourceAsReader("mybatis-config.xml");
            sqlSessionFactory = new SqlSessionFactoryBuilder().build(reader);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static SqlSession getSqlSession() {
        return sqlSessionFactory.openSession(); // 새로운 SqlSession의 인스턴스 리턴
    }
}
```



test/org.scoula.database.MyBatisConfigTest.java

```
package org.scoula.database;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class MyBatisConfigTest {
    @Test
    void testConnection() {
        assertNotNull(MyBatisConfig.getSqlSession(), "MyBatis SqlSession should not be null");
    }
}
```

✓ Mapper와 Dao

○ Mapper

- Dao에서 실행할 SQL문을 XML 엘리먼트로 정의
- namespace 속성지정 : Dao 인터페이스의 전체 클래스명으로 지정(관례)
namespace="org.scoula.travel.dao.TravelDao"
- 각 엘리먼트는 id 속성 설정
- 엘리먼트의 식별
 - namespace + 엘리먼트 id

○ Mapper와 Dao의 연동

- Dao 인터페이스의 시그니처에 맞춰 Mapper의 엘리먼트 속성 설정
 - 메서드명 → 엘리먼트 id (관례)
 - 리턴 타입 → resultType
 - 매개변수 → parameterType
1개 만 지정 가능
primitive 타입
모델 객체 또는 Map : 프로퍼티 → #{프로퍼티명} 으로 지정

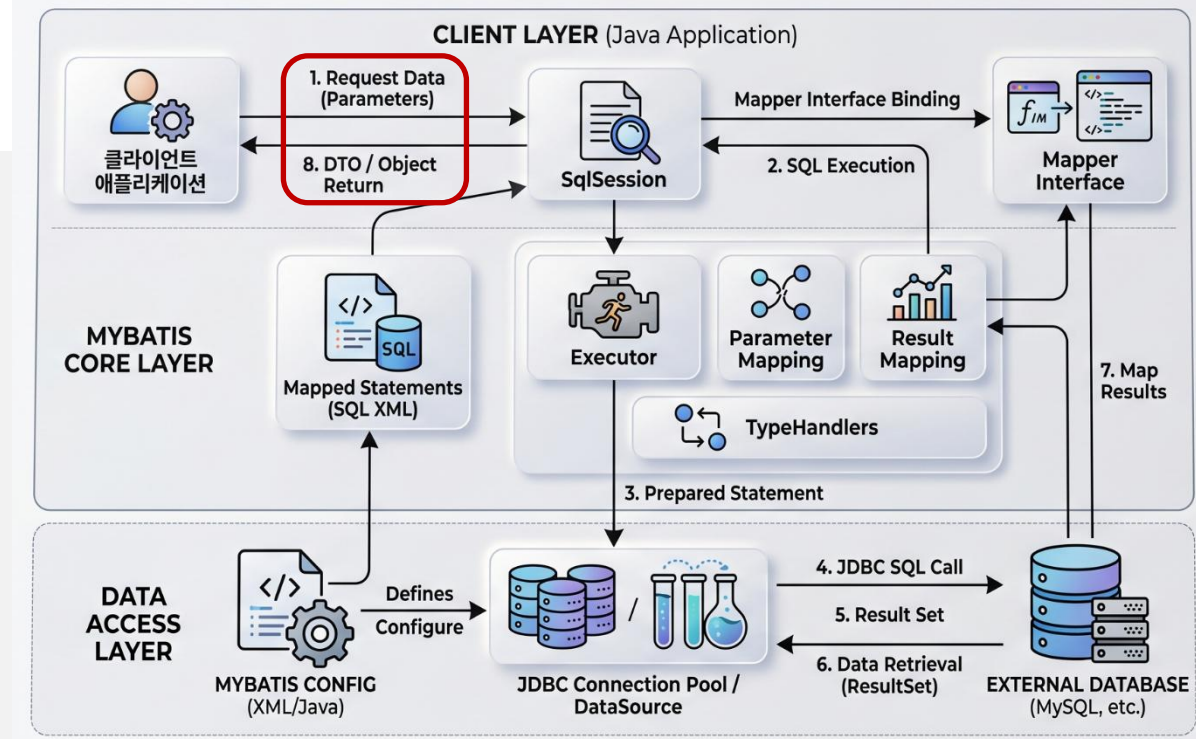
org.scoula.travel.domain.TravelVO.java

```
package org.scoula.travel.domain;
```

```
import lombok.AllArgsConstructor;  
import lombok.Builder;  
import lombok.Data;  
import lombok.NoArgsConstructor;
```

```
import java.util.List;
```

```
@Data  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class TravelVO {  
    private Long no;  
    private String district;  
    private String title;  
    private String description;  
    private String address;  
    private String phone;  
}
```



resources/mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">

<configuration>

    <properties resource="application.properties"/>

    <settings>...</settings>

    <typeAliases>
        <typeAlias type="org.scoula.travel.domain.TravelVO" alias="TravelVO"/>
    </typeAliases>

    <environments default="development">...</environments>

</configuration>
```

org.scoula.travel.dao.TravelDao.java

```
package org.scoula.travel.dao;

import org.scoula.travel.domain.TravelVO;
import java.util.List;
import java.util.Optional;

public interface TravelDao {

    int getTotalCount();

    // 페이지별 목록 얻기
    List<TravelVO> getTravels(int start, int end);

    List<String> getDistricts();    // 구역 목록 얻기

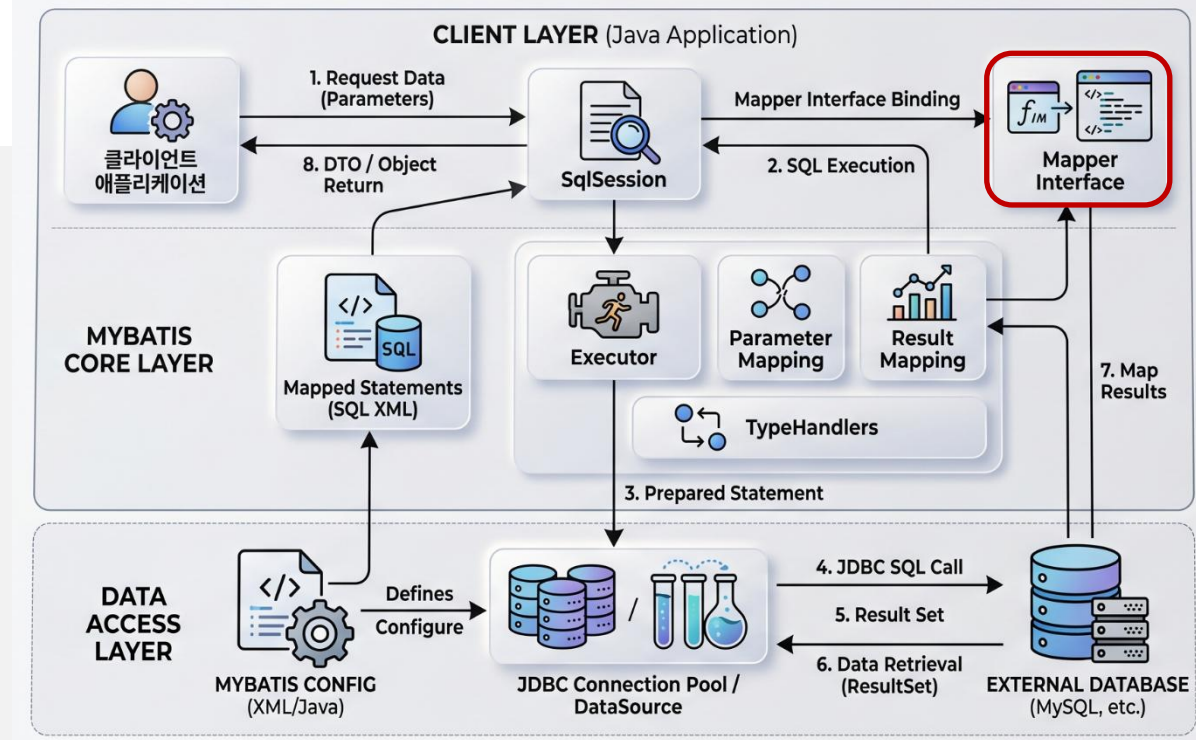
    List<TravelVO> getTravelsByDistrict(String district); // 해당 구역의 목록 얻기

    Optional<TravelVO> getTravel(Long no);    // 특정 관광지 정보 얻기

    void insert(TravelVO travel);

    void update(TravelVO travel);

    void remove(Long no);
}
```



org.scoula.travel.dao.TravelDaoImpl.java

```
package org.scoula.travel.dao;

import org.apache.ibatis.session.SqlSession;
import org.scoula.database.MyBatisConfig;
import org.scoula.travel.domain.TravelVO;

import java.util.List;
import java.util.Optional;

public class TravelDaoImpl implements TravelDao {
    static final String NAMESPACE= "org.scoula.travel.dao.TravelDao.";

    ...
}
```

✓ Mapper

- `#{프로퍼티명}` 해석, 예 : `#{userId}`
 - 인자가 프리미티브 또는 String 인 경우(단일값)
 - 프로퍼티 명과 관계 없이 지정
 - 인자가 모델 객체인 경우
 - getter 호출
`getUserId();`
 - 인자가 Map인 경우
 - `get()` 호출
`get("userId")`

resources/mapper/travel-mapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="org.scoula.travel.dao.TravelDao">
  <select id="..." ...><![CDATA[      ]]></select>
  <insert id="..." ...><![CDATA[      ]]></insert>
  <update id="..." ...><![CDATA[      ]]></update>
  <delete id="..." ...><![CDATA[      ]]></delete>
</mapper>
```

○ namespace

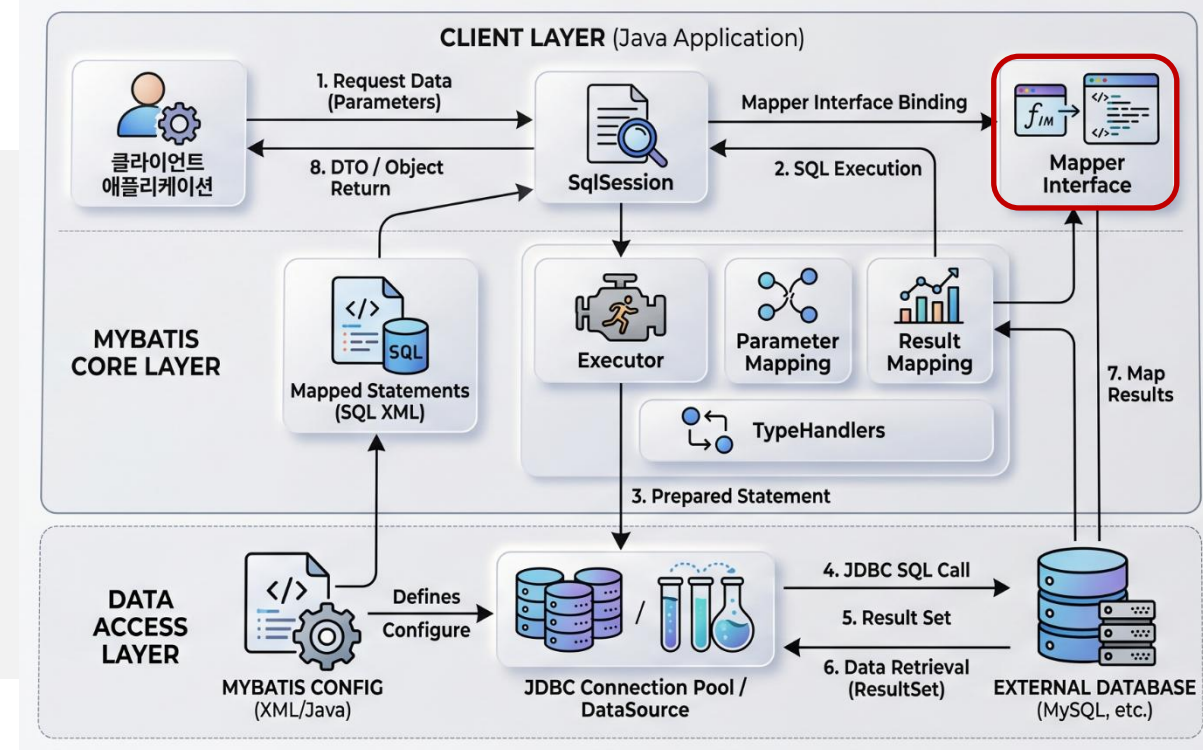
- 매핑할 전체 인터페이스 명

○ id

- 매핑할 인터페이스의 메서드 명

○ <![CDATA[]]>

- xml 표준 구문
- 이미 컴파일한 데이터이므로 단순 텍스트로 해석하라는 지시
→ 태그 특수 기호(<, > 등)를 데이터러 취급



resources/mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">

<configuration>

    <properties resource="application.properties"/>

    <settings>...</settings>

    <typeAliases>...</typeAliases>

    <environments default="development">...</environments>

    <mappers>
        <mapper resource="mapper/travel-mapper.xml"/>
    </mappers>
</configuration>
```

✓ MyBatis SqlSession CRUD 메서드

○ 단건 조회

- `<T> T selectOne(String statement);`
- `<T> T selectOne(String statement, Object parameter);`

○ 다건 조회(List)

- `<E> List<E> selectList(String statement);`
- `<E> List<E> selectList(String statement, Object parameter);`

✓ MyBatis SqlSession CRUD 메서드

○ 삽입

- `int insert(String statement);`
- `int insert(String statement, Object parameter);`

○ 수정

- `int update(String statement);`
- `int update(String statement, Object parameter);`

○ 삭제

- `int delete(String statement);`
- `int delete(String statement, Object parameter);`

✓ MyBatis SqlSession CRUD 메서드

○ 커밋

- `void commit();`
- `void commit(boolean force);`

○ 롤백

- `void rollback();`
- `void rollback(boolean force);`

✓ select 문 처리

- `int getTotalCount();` // 전체 개수 얻기

resources/mapper/travel-mapper.xml

```
<select id="getTotalCount" resultType="int">
  <![CDATA[
    SELECT COUNT(*)
    FROM tbl_travel
  ]]>
</select>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public int getTotalCount() {
    return MyBatisConfig.getSqlSession()
        .selectOne(NAMESPACE + "getTotalCount");
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
class TravelDaoImplTest {
    TravelDao dao = new TravelDaoImpl();

    @Test
    void getTotalCount() {
        int count = dao.getTotalCount();
        Assertions.assertTrue(count > 0, "Total count should be greater than 0");
        System.out.printf("Total count: %d\n", count);
    }
    ...
}
```

Total count: 112

✓ select 문 처리

○ `List<TravelVO> getTravels(int start, int end);` `// 목록 얻기`

`resources/mapper/travel-mapper.xml`

```
<select id="getTravelList" resultType="TravelVO">
  <![CDATA[
    SELECT *
    FROM tbl_travel
    ORDER BY district, title
    limit #{start}, #{size}
  ]]>
</select>
```

`org.scoula.travel.dao.TravelDaoImpl.java`

```
@Override
public List<TravelVO> getTravels(int page, int size) {
    int start = (page - 1) * size;
    Map map = Map.of("start", start, "size", size);
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getTravels", map);
}
```


test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void getTravels() {
    List<TravelVO> list = dao.getTravels(1, 10); // 1페이지, 한 페이지당 10개
    Assertions.assertTrue(list.size() > 0);
    list.forEach(System.out::println);
}
```

```
TravelVO(no=101, district=강원권, title=강릉 경포 해변, description=동해안 최대 해변으로 유명하며 강문동, 인
TravelVO(no=105, district=강원권, title=강릉 솔향수목원, description=강릉솔향수목원은 금강소나무 원시림을 간
TravelVO(no=100, district=강원권, title=강릉 연곡해변캠핑장, description=연곡해변캠핑장에서는 탁 트인 바다
TravelVO(no=106, district=강원권, title=강릉 정동진 모래시계공원, description=모래시계공원은 동해 바다와 산
TravelVO(no=103, district=강원권, title=강릉 커피거리, description=강원도 강릉시는 국내 지방자치단체 단위에
TravelVO(no=104, district=강원권, title=강릉 통일공원, description=세계유일의 분단국이며, 분단의 허리 강원도
TravelVO(no=102, district=강원권, title=강릉 허균?허난설헌 기념공원, description=강원도 강릉에 있는 허난설헌
TravelVO(no=107, district=강원권, title=동해 망상해수욕장, description=망상해변은 묵호항에서 북쪽으로 3km쯤
TravelVO(no=98, district=강원권, title=속초 아바이마을, description=청호동에 형성된 아바이마을은 실향민의 0
TravelVO(no=99, district=강원권, title=속초 해수욕장관광지, description=1976년 처음 개장한 속초해수욕장은
```

✓ select 문 처리

- `List<String> getDistricts();` // 권역 목록 얻기

`resources/mapper/travel-mapper.xml`

```
<select id="getDistricts" resultType="String">
  <![CDATA[
    SELECT DISTINCT district
    FROM tbl_travel
    ORDER BY district
  ]]>
</select>
```

`org.scoula.travel.dao.TravelDaoImpl.java`

```
@Override
public List<String> getDistricts() {
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getDistricts");
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void getDistricts() {
    List<String> list = dao.getDistricts();
    Assertions.assertTrue(list.size() > 0);
    list.forEach(System.out::println);
}
```

강원권
경상권
수도권
전라권
제주권
충청권

✓ select 문 처리

- `List<TravelVO> getTravelsByDistrict(String district);` // 해당 권역의 목록 얻기

resources/mapper/travel-mapper.xml

```
<select id="getTravelsByDistrict" resultType="TravelVO">
  <![CDATA[
    SELECT *
    FROM tbl_travel
    WHERE district = #{district}
    ORDER BY district, title
  ]]>
</select>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public List<TravelVO> getTravelsByDistrict(String district) {
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getTravelsByDistrict", district);
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void getTravelsByDistrict() {
    List<TravelV0> list = dao.getTravelsByDistrict("강원권");
    Assertions.assertTrue(list.size() > 0);
    list.forEach(System.out::println);
}
```

TravelV0(no=101, district=강원권, title=강릉 경포 해변, description=동해안 최대 해변으로 유명하며 강문동, 안현동에 있고 시내에서 북으로 6km, 경
TravelV0(no=105, district=강원권, title=강릉 솔향수목원, description=강릉솔향수목원은 금강소나무 원시림을 간직한 칠성산 자락, 강릉에서 경치 좋
TravelV0(no=100, district=강원권, title=강릉 연곡해변캠핑장, description=연곡해변캠핑장에서는 탁 트인 바다 전망과 풍성한 소나무 군락을 동시에 누
TravelV0(no=106, district=강원권, title=강릉 정동진 모래시계공원, description=모래시계공원은 동해 바다와 산, 정동진 2리 마을을 배경으로 아담하
TravelV0(no=103, district=강원권, title=강릉 커피거리, description=강원도 강릉시는 국내 지방자치단체 단위에서는 최초로 커피 축제를 개최한 곳으로
TravelV0(no=104, district=강원권, title=강릉 통일공원, description=세계유일의 분단국이며, 분단의 허리 강원도.강원도 해안 중심도시인 강릉 안인진
TravelV0(no=102, district=강원권, title=강릉 허균?허난설헌 기념공원, description=강원도 강릉에 있는 허난설헌 생가터는 조선 시대 최고의 여류문인
TravelV0(no=107, district=강원권, title=동해 망상해수욕장, description=망상해변은 묵호항에서 북쪽으로 3km쯤 떨어져 있는 국민관광지로서 부근의 모
TravelV0(no=98, district=강원권, title=속초 아바이마을, description=청호동에 형성된 아바이마을은 실향민의 아픔이 진하게 배어나는 곳이다. 6.25 이
TravelV0(no=99, district=강원권, title=속초 해수욕장관광지, description=1976년 처음 개장한 속초해수욕장은 국내 대표적인 해수욕장 가운데 하나다.
TravelV0(no=108, district=강원권, title=정선 삼탄아트마인, description=삼탄아트마인은 1964년에서 2001년까지 삼척탄좌가 있던 자리다. 그 터 위에
TravelV0(no=95, district=강원권, title=춘천 남이섬, description=남이섬은 [동화나라, 노래의섬]을 컨셉으로 다양한 문화행사, 콘서트 및 전시를 꾸준
TravelV0(no=92, district=강원권, title=춘천 박사마을 어린이 글램핑장, description=춘천시가 의암호 수변에 조성한 캠핑 시설로 글램핑장과 오토캠핑?

✓ select 문 처리

- `Optional<TravelVO> getTravel(Long no);` // 특정 관광지 정보 얻기

resources/mapper/travel-mapper.xml

```
<select id="getTravel" resultType="TravelVO">
  <![CDATA[
    SELECT *
    FROM tbl_travel
    WHERE no = #{no}
  ]]>
</select>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public Optional<TravelVO> getTravel(Long no) {
    TravelVO travelVO = MyBatisConfig.getSqlSession()
        .selectOne(NAMESPACE + "getTravel", no);
    return Optional.ofNullable(travelVO);
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void getTravel() {
    TravelV0 travel = dao.getTravel(1L).orElseThrow();
    Assertions.assertNotNull(travel);
    System.out.println(travel);
}
```

TravelV0(no=1, district=수도권, title=인천 개항장역사문화공간, description=1883년 인천 개항장 126년 세월의 흔적이 쌓인 개항기 역사와 문화의 만남의 장소이다. 이곳은 옛 일본영사관인 중구청과 일본 제1은행 인천지점, 일본 제18은행 인천지점, 일본 제58은행 인천지점 등 근대 건축을 그대로 볼 수 있는 장소이다. 이뿐만아니라 일본우선주식회사나 인천의 아트플랫폼, 제물포구락부 등 역사의 흔적들을 직접 눈으로 볼 수 있고 귀로 들을 수있는 곳이다. 이곳은 인천문화 관광해설사가 배치되어 예약을 받아 해설을 하고 있다. 보다 깊은 해설을 들으며 도보관광을 즐길 수 있어 가족 단위나 여행단체의 큰 호응을 얻고 있다., address=인천광역시 중구 제물량로232번안길 27, phone=인천 중구 문화유산팀 032-760-6448)

☑ 트랜잭션 처리

○ MyBatis는 Auto Commit이 디폴트로 비활성화 되어 있음.

- insert, update, delete 호출 시 명시적으로 commit() 호출 필요
- try-with-resource를 사용하는 경우 commit()이 호출되지 않은 상태로 close()가 실행되면
 - DB 레벨에서 진행 중이던 트랜잭션을 취소(Rollback)함
 - 에러가 나면 그냥 예외를 던지기만 해도 안전하게 롤백이 보장됨

```
try (SqlSession session = MyBatisConfig.getSqlSession()) {  
    트랜잭션 연산...  
    session.commit();  
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}
```

→ Consumer<SqlSession>을 이용하여 공통 코드 분리 가능

```
try (SqlSession session = MyBatisConfig.getSqlSession()) {  
    consumer.accept(session);  
    session.commit();  
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}
```


test/org.scoula.travel.dao.TravelDaoImpl.java

```
private void transaction(Consumer<SqlSession> action) {  
    try (SqlSession session = MyBatisConfig.getSqlSession()) {  
        action.accept(session);  
        session.commit();  
    }  
}
```

○ MyBatis 예외 클래스

- PersistenceException
 - MyBatis의 최상위 예외 객체
 - RuntimeException으로 예외처리가 필수는 아님

✓ insert 문 처리

- void insert(TravelVO travel);

resources/mapper/travel-mapper.xml

```
<insert id="insert" parameterType="TravelVO">
  <![CDATA[
    INSERT INTO tbl_travel (district, title, description, address, phone)
    VALUES (#{district}, #{title}, #{description}, #{address}, #{phone})
  ]]>
</insert>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public void insert(TravelVO travel) {
  transaction(session -> session.insert(NAMESPACE + "insert", travel));
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
void insert() {  
    TravelV0 travel = TravelV0.builder()  
        .district("수도권")  
        .title("해너미 명소")  
        .description("해너미 명소로 유명해요")  
        .address("인천 광역시 서해바다")  
        .phone("111-222-3333")  
        .build();  
  
    dao.insert(travel);  
    System.out.print(travel);  
}
```

TravelV0(**no=null**, district=수도권, title=해너미 명소, description=해너미 명소로 유명해요, address=인천 광역시 서해바다, phone=111-222-3333)>

✓ update 문 처리

- void update(TravelVO travel);

resources/mapper/travel-mapper.xml

```
<update id="update" parameterType="TravelVO">
  <![CDATA[
    UPDATE tbl_travel
    SET district    = #{district},
        title       = #{title},
        description = #{description},
        address     = #{address},
        phone       = #{phone}
    WHERE no = #{no}
  ]]>
</update>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public void update(TravelVO travel) {
    transaction(session -> session.update(NAMESPACE + "update", travel));
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void update() {
    TravelVO travel = TravelVO.builder()
        .no(116L) # 실제 no 확인 후 사용
        .district("수도권")
        .title("해너미 명소___")
        .description("해너미 명소로 유명해요___")
        .address("인천 광역시 서해바다")
        .phone("111-222-3333")
        .build();

    dao.update(travel);
}
```

✓ delete 문 처리

- void remove(Long no);

resources/mapper/travel-mapper.xml

```
<delete id="remove">
  <![CDATA[
    DELETE
    FROM tbl_travel
    WHERE no = #{no}
  ]]>
</delete>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public void remove(Long no) {
    transaction(session -> session.delete(NAMESPACE + "remove", no));
}
```

test/org.scoula.travel.dao.TravelDaoImplTest.java

```
@Test
void remove() {
    Long no = 116L;
    dao.remove(no);
}
}
```

resources/mapper/travel-mapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="org.scoula.travel.dao.TravelDao">
    <select id="getTotalCount" resultType="int">
        <![CDATA[
            SELECT COUNT(*)
            FROM tbl_travel
        ]]>
    </select>

    <select id="getTravels" resultType="TravelVO">
        <![CDATA[
            SELECT *
            FROM tbl_travel
            ORDER BY district, title
            LIMIT #{start}, #{end}
        ]]>
    </select>
```


resources/mapper/travel-mapper.xml

```
<select id="getDistricts" resultType="String">
    <![CDATA[
        SELECT DISTINCT district
        FROM tbl_travel
        ORDER BY district
    ]]>
</select>

<select id="getTravelsByDistrict" resultType="TravelVO">
    <![CDATA[
        SELECT *
        FROM tbl_travel
        WHERE district = #{district}
        ORDER BY district, title
    ]]>
</select>

<select id="getTravel" resultType="TravelVO">
    <![CDATA[
        SELECT *
        FROM tbl_travel
        WHERE no = #{no}
    ]]>
</select>
```

resources/mapper/travel-mapper.xml

```
<insert id="insert" parameterType="TravelVO">
    <![CDATA[
        INSERT INTO tbl_travel (district, title, description, address, phone)
        VALUES (#{district}, #{title}, #{description}, #{address}, #{phone})
    ]]>
</insert>

<update id="update" parameterType="TravelVO">
    <![CDATA[
        UPDATE tbl_travel
        SET district    = #{district},
            title       = #{title},
            description  = #{description},
            address     = #{address},
            phone       = #{phone}
        WHERE no = #{no}
    ]]>
</update>

<delete id="remove">
    <![CDATA[
        DELETE
        FROM tbl_travel
        WHERE no = #{no}
    ]]>
</delete>
</mapper>
```

org.scoula.travel.dao.TravelDaoImpl.java

```
package org.scoula.travel.dao;

import org.apache.ibatis.session.SqlSession;
import org.scoula.database.MyBatisConfig;
import org.scoula.travel.domain.TravelVO;

import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Consumer;

public class TravelDaoImpl implements TravelDao {
    static final String NAMESPACE = "org.scoula.travel.dao.TravelDao.";

    @Override
    public int getTotalCount() {
        return MyBatisConfig.getSqlSession()
            .selectOne(NAMESPACE + "getTotalCount");
    }
}
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public List<TravelVO> getTravels(int page, int size) {
    int start = (page - 1) * size;
    int end = start + size;
    Map map = Map.of("start", start, "end", end);
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getTravels", map);
}

@Override
public List<String> getDistricts() {
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getDistricts");
}

@Override
public List<TravelVO> getTravelsByDistrict(String district) {
    return MyBatisConfig.getSqlSession()
        .selectList(NAMESPACE + "getTravelsByDistrict", district);
}
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public Optional<TravelV0> getTravel(Long no) {
    TravelV0 travelV0 = MyBatisConfig.getSqlSession()
        .selectOne(NAMESPACE + "getTravel", no);
    return Optional.ofNullable(travelV0);
}

private void transaction(Consumer<SqlSession> action) {
    try (SqlSession session = MyBatisConfig.getSqlSession()) {
        action.accept(session);
        session.commit();
    }
}

@Override
public void insert(TravelV0 travel) {
    transaction(session -> session.insert(NAMESPACE + "insert", travel));
}

@Override
public void update(TravelV0 travel) {
    transaction(session -> session.update(NAMESPACE + "update", travel));
}
```

org.scoula.travel.dao.TravelDaoImpl.java

```
@Override
public void remove(Long no) {
    transaction(session -> session.delete(NAMESPACE + "remove", no));
}

}
```